



INSTITUTO DE EDUCACIÓN SECUNDARIA JOSÉ PLANES

**Departamento de Informática y Comunicaciones
Técnico Superior en Desarrollo de aplicaciones Web**

C/ Maestro Pérez Abadía, 2

30100 Espinardo – Murcia

T. 968 834 605

30010577@murciaeduca.es

www.iesjoseplanes.es

Memoria del proyecto

Desarrollo de aplicaciones web

Iven Project

Autores/as:

Mehdi Ziad El Amrani

Profesor/a-coordinador/a:

Susana Valverde

Fernando Ruiz

Murcia, Junio de 2026



Esta obra está bajo una [licencia de Creative Commons Reconocimiento-CompartirIgual 4.0 Internacional](https://creativecommons.org/licenses/by-sa/4.0/).

A nuestras familias por su constante apoyo y paciencia durante estos dos años de formación. A los profesores del Departamento de Informática del IES José Planes por brindarnos las herramientas de conocimiento necesarias, guiarnos en el proceso de desarrollo y exigirnos calidad en cada entrega. Por último, a nuestros compañeros de clase, con quienes compartimos incontables horas de código, depuración y aprendizaje colaborativo.

Contenido

1	Resumen extendido	1
1.1	Kjhkjh	1
2	Palabras clave	1
3	Introducción	1
4	Estado del arte/trabajos relacionados	1
5	Análisis de objetivos y metodología.	1
6	Diseño y resolución del trabajo realizado	2
7	Presupuesto	2
8	Conclusiones y vías futuras	2
9	Bibliografía/Webgrafía.	2

1 Resumen extendido

Iven es un sistema web full-stack diseñado específicamente para dar solución al control del equipamiento informático en centros educativos de educación secundaria. En la práctica real, los institutos acumulan centenares de ordenadores de sobremesa, portátiles, monitores, teclados y ratones que se mueven constantemente entre aulas de informática, laboratorios de ciclos formativos, departamentos didácticos y oficinas administrativas. Iven permite agrupar estos componentes físicos individuales en estaciones de trabajo unificadas denominadas Products (o bundles), facilitando su administración centralizada, trazabilidad física e incidencias asociadas.

La aplicación consta de una API RESTful desarrollada sobre Java 21 y Spring Boot 3.4.5, apoyándose en la base de datos relacional PostgreSQL para la persistencia de datos. Por otro lado, la interfaz de usuario se ha implementado utilizando Angular 21 con componentes autónomos (standalone components), reactividad basada en Signals, control de acceso con interceptores JWT y pruebas unitarias con Vitest. El sistema ofrece un portal administrativo que permite dar de alta periféricos y ordenadores, agruparlos en puestos informáticos funcionales, trasladar estos puestos físicamente registrando un histórico cronológico, y reportar incidencias asociadas a los mismos, todo bajo una interfaz de alta fidelidad que refleja la identidad cromática amarilla y negra del IES José Planes.

1.1 Kjhkjh

Iven introduce el concepto de "puesto ensamblado" o Product, asociando un ordenador (Computer), una pantalla (Monitor), un teclado (Keyboard) y un ratón (Mouse) a un identificador lógico único. Al realizar un movimiento de aula, o al registrar una avería ("Pantalla parpadea"), la acción no se realiza de forma atómica e inconexa, sino que afecta al puesto completo, simplificando la toma de decisiones del coordinador de tecnologías y agilizando las labores de mantenimiento.

2 Palabras clave

Inventario escolar, Angular 21, Spring Boot 3.4.5, Java 21, TypeScript, PostgreSQL, códigos QR, gestión de incidencias, estaciones de trabajo, JSON Web Tokens (JWT), Reactividad, Signals, REST API, Spring Security, Docker, Vitest, IES José Planes.

3 Introducción

3.1. Origen de la idea

El proyecto surgió de la necesidad percibida por los alumnos del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web al observar la gestión de los recursos informáticos en el IES José Planes. El departamento de informática y la coordinación de TIC del centro debían lidiar frecuentemente con hojas de cálculo estáticas para saber qué ordenadores estaban asignados a cada aula, qué periféricos se habían averiado o qué especificaciones técnicas (RAM, procesador) tenían las máquinas en cada laboratorio. El descontrol de versiones, la falta de concurrencia y la nula trazabilidad histórica de los traslados de hardware hacían que la gestión fuera ineficiente. Iven nace para sustituir las hojas de cálculo por una plataforma web segura, interactiva y multiusuario.

3.2. Contexto y problemática

Los institutos de enseñanza secundaria en la Región de Murcia cuentan con múltiples aulas equipadas con ordenadores de diversas generaciones, marcas y proveedores, adquiridos a lo largo de diferentes programas de digitalización de la Consejería de Educación. Cuando un componente de hardware se avería, suele quedar abandonado en un aula o almacenado sin registrar el motivo. Cuando un profesor traslada un monitor de un aula a otra, la información no se actualiza. A final de curso, realizar el inventario requiere semanas de trabajo manual. Iven soluciona esto permitiendo una catalogación pormenorizada de

cada periférico e implementando un sistema dinámico de traslados físicos y gestión de estado de incidencias.

4 Estado del arte/trabajos relacionados

Existen sistemas consolidados en el sector de la gestión de activos informáticos y de soporte técnico:

1. Snipe-IT: Herramienta open-source muy potente escrita en PHP (Laravel) para la gestión de activos empresariales. Aunque es el referente de la industria, resulta excesivamente compleja para un entorno escolar. Requiere la configuración de múltiples parámetros corporativos (depreciación de activos, licencias de software complejas, firmas de entrega de material) que saturan al personal no técnico de un instituto.
2. GLPI: Un sistema robusto enfocado en la gestión de servicios de TI (ITSM) que incluye un motor de inventariado y un sistema de tickets para helpdesk. Sin embargo, su interfaz gráfica clásica ha quedado obsoleta en usabilidad, y su curva de aprendizaje es sumamente empinada para un docente o administrativo de secundaria.
3. Hojas de Cálculo (Excel / Google Sheets): Es la solución por defecto en la mayoría de institutos. Es inmediata pero peligrosa: no tiene integridad referencial (se pueden escribir nombres de aulas inexistentes), no gestiona históricos de ubicaciones de manera nativa, no restringe accesos por roles administrativos y carece de integración con flujos de reparación de incidencias.

Valor añadido de Iven:

Iven se enfoca directamente en la realidad de los centros educativos públicos de Murcia. A diferencia de las herramientas corporativas pesadas, introduce un flujo simplificado adaptado a la organización física típica de un IES (aulas, departamentos y laboratorios). Su valor añadido reside en:

* El modelo de empaquetado (Workstation/Product Bundle) que une ordenadores y periféricos en una sola entidad.

* Una interfaz moderna basada en la identidad visual del instituto IES José Planes.

* Un sistema de generación de códigos QR integrado de forma nativa para permitir a los técnicos de soporte acceder instantáneamente a la ficha del puesto escaneando la etiqueta pegada en la mesa del aula.

5 Análisis de objetivos y metodología.

A continuación se analizan los objetivos técnicos planteados para el desarrollo del proyecto, contrastándolos con la realidad de su implementación y las tecnologías empleadas.

* OBJETIVO 1. Diseñar un modelo de datos estructurado y normalizado

Resolución: Se diseñó un esquema relacional con PostgreSQL. Se crearon tablas específicas para periféricos (computers, monitors, keyboards, mice) y entidades de clasificación (item_types, brands, suppliers). Para evitar el desorden, se implementó la tabla products que unifica los cuatro periféricos principales bajo una clave primaria.

Dificultades: La principal dificultad radicó en coordinar la relación de las incidencias y las ubicaciones. Inicialmente se consideró una relación directa sobre cada periférico. Sin embargo, tras analizar el caso de uso real de las aulas, se optó por asociar tanto las incidencias (incidencias) como la geolocalización (product_locations) a la entidad agrupadora Product, simplificando enormemente las consultas y la lógica de negocio.

* OBJETIVO 2. Implementar un backend robusto y moderno

Resolución: Se desarrolló un servidor REST con Java 21 y Spring Boot 3.4.5. Se utilizó Spring Security y JSON Web Tokens (JWT) para asegurar los endpoints de la API, distinguiendo entre roles de lectura (ROLE_USER) y roles de administración (ROLE_ADMIN). Las contraseñas se almacenan cifradas en la base de datos utilizando el algoritmo de hashing BCrypt.

Dificultades: Configurar la cadena de filtros en las versiones recientes de Spring Security 3.x supuso un reto técnico debido a la deprecación de los métodos de configuración clásicos en favor del uso de expresiones lambda y DSLs de seguridad.

* OBJETIVO 3. Desarrollar una interfaz de usuario reactiva y eficiente

Resolución: Se construyó el frontend en Angular 21 utilizando standalone components para eliminar el acoplamiento de los módulos tradicionales. Se emplearon Signals como motor principal de estado reactivo de la aplicación en componentes clave como la lista de inventario global y el panel de estadísticas.

Dificultades: El uso de la nueva versión de Angular 21 y TypeScript 5.9 supuso una investigación minuciosa de las APIs nativas de Signals, reemplazando el uso intensivo de observables de RxJS por señales para mejorar el rendimiento del renderizado en el navegador.

* OBJETIVO 4. Gestionar ubicaciones físicas y traslados

Resolución: Cada puesto informático (Product) puede ser trasladado dinámicamente de ubicación. Para ello se implementó la entidad ProductLocation, la cual registra el nombre de la ubicación, el tipo (aula, laboratorio, almacén, oficina), la fecha del movimiento, el usuario que lo realizó y notas adicionales. Se configuró un flag booleano is_current para determinar la ubicación activa actual, manteniendo el resto de registros como histórico de movimientos del puesto.

* OBJETIVO 5. Implementar identificación rápida mediante códigos QR

Resolución: Se integró la librería open-source qrcode en el frontend Angular. El sistema genera una URL única para cada puesto del tipo `http://[host]/[tipo-item]/[id]` y la codifica en un código QR dinámico. El usuario puede visualizarlo en el detalle del equipo y descargarlo en formato PNG para imprimirlo y pegarlo físicamente sobre los equipos.

* OBJETIVO 6. Diseñar un panel de control intuitivo

Resolución: Se creó un dashboard interactivo utilizando señales computadas en Angular 21 que muestra de forma gráfica la distribución de equipos por tipo, estado, incidencias activas clasificadas por prioridad (baja, media, alta, crítica) y actividad reciente. En lugar de sobrecargar la aplicación con dependencias externas como Chart.js, se optó por diseñar gráficos de barras visuales nativos y responsivos haciendo uso de maquetación CSS moderna y directivas condicionales de Angular (@for y @switch).

* OBJETIVO 7. Utilizar bases de datos no relacionales (Descartado)

Resolución: Durante la fase de diseño inicial se sopesó la utilización de MongoDB (base de datos documental) para almacenar las fichas de los equipos informáticos debido a su flexibilidad de esquema. Sin embargo, esta opción fue descartada.

Justificación: El inventario de un centro educativo requiere una estricta consistencia referencial: no podemos permitir que un equipo apunte a un proveedor inexistente o que un traslado se registre en un aula borrada. Las transacciones ACID y las claves foráneas de PostgreSQL garantizan la integridad de los datos de manera mucho más sólida que un sistema NoSQL, donde las relaciones polimórficas habrían tenido que ser gestionadas manualmente en el código de la aplicación.

6 Diseño y resolución del trabajo realizado

6.1. Esquema de Base de Datos

La base de datos se implementó en PostgreSQL. El esquema está normalizado e incluye relaciones relacionales sólidas:

(El diagrama de relaciones completo ha sido renderizado en formato de texto plano dentro del archivo del proyecto).

6.2. Arquitectura de Software del Backend

El backend de Spring Boot se estructuró siguiendo una arquitectura de tres capas clásica, desacoplada mediante objetos de transferencia de datos (DTOs) y mapeadores manuales:

1. Capa del Modelo (Entities): Representan las tablas de la base de datos mapeadas con anotaciones de Jakarta Persistence (JPA).
2. Capa de Repositorio: Interfaces que heredan de JpaRepository, proporcionando las operaciones CRUD básicas y consultas personalizadas mediante métodos derivados de Spring Data.
3. Capa de Controlador (RestController): Clases que interceptan las peticiones HTTP, validan los cuerpos de las solicitudes utilizando @Valid y devuelven respuestas en formato JSON a través de DTOs.

6.3. Arquitectura y Lógica del Frontend (Angular 21)

El frontend implementa una SPA (Single Page Application) altamente estructurada. Al hacer uso de Angular 21, todos los componentes importan sus módulos e interfaces explícitamente:

- * Gestión del Estado mediante Signals: Se prescinde de variables de componente tradicionales, reduciendo la necesidad de ejecutar ciclos de detección de cambios de manera global. Las señales garantizan que el DOM solo se actualice de forma selectiva en los nodos enlazados a la señal modificada.
- * Componente de Visualización Reactiva (AllItemsComponent): El listado global descarga ordenadores, monitores, teclados y ratones en paralelo mediante la función forkJoin de RxJS, unifica las listas en un modelo común plano (FlatItem) y expone opciones dinámicas de filtrado reactivo en tiempo de ejecución.

7 Presupuesto

7.1. Recursos Humanos

El desarrollo de Iven se ha estimado simulando tarifas de desarrollo freelance junior del mercado español. El proyecto fue realizado por el equipo de alumnos durante el periodo de prácticas y módulo de proyecto.

| Fase del Proyecto | Horas Estimadas | Descripción del trabajo realizado |

|---|---|---|

| Análisis y Diseño del Modelo | 25 h | Requisitos del IES Planes, diagramas de base de datos y mockups. |

| Desarrollo de API Rest (Backend) | 55 h | Creación de endpoints de periféricos y puestos, seguridad JWT y lógica de traslados. |

| Desarrollo de Panel Web (Frontend) | 60 h | Maquetación responsive, integración de Signals de Angular 21, CRUDs y QR. |

| Fase de Testing y QA | 20 h | Pruebas unitarias con Vitest y validación de endpoints. |

| Documentación y Despliegue | 15 h | Redacción de memoria y contenedorización con Docker. |

| TOTAL | 175 h | |

7.2. Costes Económicos

Suponiendo un precio medio de la hora de desarrollo junior en la Región de Murcia de 25 €/hora:

* Coste del Personal de Desarrollo: 175 horas × 25 €/hora = 4.375,00 €

* Hosting y Servidor en la Nube (VPS de 1 año para producción): 120,00 €

* Dominio Escolar (.es): 15,00 €

* TOTAL ESTIMADO DEL PROYECTO: 4.510,00 €

(Esto en ámbito profesional y en un contexto en el que se hubiera hecho un seguimiento real del proyecto.)

8 Conclusiones y vías futuras

8.1. Conclusiones

El desarrollo de Iven demuestra que la arquitectura modular y la normalización relacional son fundamentales antes de empezar a picar código. La decisión de empaquetar periféricos en la entidad abstracta de Product nos permitió resolver de forma limpia el seguimiento físico del equipamiento de las aulas, evitando consultas duplicadas e inconsistencias en la base de datos.

El uso de tecnologías modernas como Java 21 y Angular 21 no solo mejoró el rendimiento global de la aplicación (gracias al motor reactivo de Signals), sino que enriqueció nuestro perfil profesional dotándonos de experiencia práctica en versiones avanzadas y demandadas del ecosistema de desarrollo de software.

8.2. Dificultades superadas

- * Reactividad en el Dashboard: La sincronización de múltiples contadores y estadísticas en tiempo real se resolvieron usando reactividad nativa. La gestión de errores individuales con `catchError` en las consultas unificadas garantizó que fallos en un periférico no congelen todo el dashboard del usuario.
- * Seguridad y Roles: Controlar el acceso seguro a través de tokens JWT, almacenando las cabeceras correctamente y configurando los guards de Angular para restringir las operaciones de alta y baja exclusivamente a perfiles de administración (`ROLE_ADMIN`).
- * Generación de códigos QR de alta resolución: Resolver la transformación de enlaces lógicos a imágenes escaneables en dispositivos móviles asegurando compatibilidad cruzada en navegadores web mediante la conversión a URLs de tipo Data (Base64).

8.3. Habilidades Blandas (Soft Skills) Desarrolladas

* Trabajo en equipo: Planificar tareas y evitar conflictos de Git mediante el uso estricto de ramas y revisiones de código de forma colaborativa.

* Investigación Autónoma: Localizar documentación y solventar las diferencias de la API de seguridad entre Spring Security 5 y Spring Security 6/3.x.

8.4. Vías Futuras de Desarrollo

El sistema está listo para su despliegue, pero cuenta con amplios horizontes de mejora:

1. Lector de Códigos QR integrado por cámara: Implementar de forma nativa la lectura de las etiquetas impresas desde la cámara del smartphone haciendo uso de la librería html5-qrcode que ya se ha incorporado en las dependencias de la aplicación.

2. Generación automática de backups: Crear scripts programados que extraigan volcados lógicos de PostgreSQL y los almacenen automáticamente en unidades externas de almacenamiento.

3. Notificaciones de incidencias: Integrar servicios de correo (JavaMailSender) para avisar al administrador cuando un profesor registre una incidencia con prioridad "Crítica" o "Alta".

9 Bibliografía/Webgrafía.

1. Angular. (2024). Angular Documentation - Signals in Angular 21. Recuperado de <https://angular.dev/guide/signals>

2. Oracle. (2023). Java SE 21 Platform Specification. Recuperado de <https://docs.oracle.com/en/java/javase/21/>

3. Spring Boot. (2024). Spring Boot Reference Guide v3.4.5. Recuperado de <https://docs.spring.io/spring-boot/docs/current/reference/html/>

4. Davidshimjs. (2023). QRCode.js - Cross-browser QRCode generator for Javascript. GitHub Repository. Recuperado de <https://github.com/davidshimjs/qrcodejs>

5. PostgreSQL Global Development Group. (2024). PostgreSQL 15 Documentation. Recuperado de <https://www.postgresql.org/docs/15/>
6. Playwright. (2024). Playwright Test Runner Documentation. Recuperado de <https://playwright.dev/docs/intro>
7. Vitest. (2024). Vitest - Blazing Fast Unit Test Framework. Recuperado de <https://vitest.dev/>
8. IES José Planes. (2024). Instrucciones y plantillas para la presentación de la memoria del módulo de Proyecto. Murcia: IES José Planes.